What computation becomes unnecessary?

The mirror box: uncertainty, recognition and closure

Endless Monkey Project
Relational Geometry Program
November 10, 2026 · manuscript for independent review · version 0.3

Abstract

When part of the uncertainty is already resolved by inherited relations, recognizing it saves computation, and recognizing it also costs something. This manuscript follows that account inside the mirror box, in four steps. In the tetrahedron, six comparisons between four states are rebuilt from three, because a comparison has no direction. As uncertainty shrinks, five of eight states are already enough to recognize the only four-state closure that fits, while one state stays pending; the next exclusion either closes or undoes what was recognized. When the reflections are given as parity equations, the exact number of solutions costs $O(\mathrm{poly}(L)(1+2^{\min(t,d)}))$, where d is the uncertainty the closure leaves free and t is the remaining novelty. And when the reflections arrive hidden inside logic gates, recognizing them has a cost of its own. We measure it against a DPLL that enumerates and against variable elimination on the formula as it arrives, at increasing sizes. On constant-width formulas, both exact routes grow polynomially and differ by a constant factor. On formulas with long connections, the largest elimination table grows from 2^6 to 2^{58}, while recognition counts 256 primary variables in 58 ms. Each result holds for the family its section defines; the question for three-literal formulas without recognizable templates stays open in §12.

Keywords: uncertainty, comparison, reflection, closure, inheritance, novelty, recognition, exact counting.

1. The question

The program's November 8 paper, *Where does computation become cheaper?*, separated three things: how many differences a query sees, how many must be kept to rebuild the answer, and how many operations it takes to give it. This manuscript continues it with a fourth: how much it costs to recognize which part of the uncertainty was already resolved.

The mirror box is the conceptual laboratory. A mirror reflects one state in another. A closure is a set that contains the reflection of its own combinations. A new constraint moves the wall: either it lands on a reflection that was already there, and then it is inheritance, or it opens a comparison that had nothing to compare with, and then it is novelty. Each image is translated into a definition and a checkable count before any saving is discussed.

The results come in four kinds: theorems under written hypotheses, exhaustive finite controls, measurements on specific implementations, and open questions.

2. Uncertainty, inheritance and novelty

Let X be a finite set of states and $\mathcal{Q}$ a family of queries. Two states are indistinguishable, $x \sim_{\mathcal{Q}} y$, when every query gives the same answer on both. The resolution of the query is

$$D_{\mathcal{Q}} = \log_2 |X/\!\sim_{\mathcal{Q}}|,$$

the number of bits of difference the query can see. It counts classes; no probability distribution is involved.

A relation is **inherited** when it already follows from what is fixed. A constraint is **novelty** when it excludes states that what is fixed still allowed. Both words always refer to a reference and a query.

The economy has two structural measures and three work measures:

1. D_{obs}: what the query needs to distinguish.
2. D_{rep}: the least that must be kept to rebuild it, within a declared class of representations.
3. C_{rec}: recognizing, checking and maintaining the reference.

4. C_{calc}: counting on that reference and carrying what is pending.
5. C_{recon}: rebuilding the answer.

The first two are measured in bits; the other three, in operations. Against a comparison route B, with the same problem, the same answer and the same cost model, there is net economy when

$$C_{\text{rec}}(F) + C_{\text{calc}}(F) + C_{\text{recon}}(F) < C_B(F).$$

Storing little and computing little are two different things: the shortest representation can be expensive to find.

3. The tetrahedron: comparison without direction

Four states $s_0, s_1, s_2, s_3 \in \mathbb{F}_2$ sit on the vertices of a tetrahedron. Each edge carries the comparison between its two ends, the reflection of one state in the other:

$$b_{ij} = s_i \oplus s_j.$$

From any vertex f, the three comparisons leaving it rebuild all six: $b_{ij} = b_{fi} \oplus b_{fj}$.

Theorem 1. The six compatible comparisons form eight patterns. Three bits encode them exactly, and three is the minimum for any injective binary code.

Proof. The linear map $\delta : \mathbb{F}_2^4 \to \mathbb{F}_2^6$, $\delta(s)_{ij} = s_i \oplus s_j$, has kernel $\{0000, 1111\}$. By rank–nullity its image has dimension 3: that is $2^3 = 8$ patterns. The three comparisons from one vertex separate them all, and two bits separate at most four. □

Flipping all four states at once leaves the six comparisons intact: a comparison records the difference, not the side it was made from. That is why the 16 vertex configurations give 8 patterns, and one bit—the orientation of the whole—lies outside what the edges see. The economy here is one of representation: three bits hold six comparisons.

4. Uncertainty shrinks: recognizing before closing

We work in $\mathbb{F}_2^n$, the 2^n states of n bits. A **closure** of dimension d is a set P of 2^d states that contains the reflection of its own combinations: if $a, b, c \in P$, then $a \oplus b \oplus c \in P$. Let S be the set of states still admitted.

Theorem 2 (only one closure fits). If $|S| < 3 \cdot 2^{d-1}$, then S contains at most one closure of dimension d.

Proof. Two distinct closures of dimension d share at most 2^{d-1} states, so together they occupy at least $2^{d+1} - 2^{d-1} = 3 \cdot 2^{d-1}$. □

The theorem gives uniqueness when a closure exists; whether one exists depends on S.

$\mathbb{F}_2^3$ has eight states: three bits of uncertainty. Each exclusion resolves part of it. After three exclusions five states remain, and in all 56 possible cases four of them form the only closure that fits; the fifth is the uncertainty that is still open. The closure can already be recognized, with one state still pending.

The fourth exclusion decides. If it resolves the pending state, the four states of the closure remain: it closes. If it removes a state of the recognized closure, that closure no longer fits: what was recognized is retracted. The exhaustive walk over the 336 ordered sequences of three exclusions and their 1,680 continuations gives 336 closures and 1,344 retractions.

The uncertainty account is exact. Going from eight states to four resolves one bit:

$$\log_2 \frac{8}{5} + \log_2 \frac{5}{4} = 1.$$

The first three exclusions resolve 0.678 bits and already allow the closure to be recognized; the fourth resolves the remaining 0.322 and confirms it. Recognition happens before closure. The pending state is kept until an exclusion resolves it, because discarding it earlier would change the answer of every query that distinguishes it.

5. Three clauses: one closure and one pending state

A three-literal clause over three distinct variables excludes one of the eight states of those variables. Let $a, b, c \in \mathbb{F}_2^3$ be three distinct states excluded by three clauses. Their own closure and its complement are

$$L = \{a, b, c, a \oplus b \oplus c\}, \qquad P = \mathbb{F}_2^3 \setminus L.$$

Theorem 3. The states admitted by the three clauses are $P \cup \{a \oplus b \oplus c\}$: a four-state closure and one pending state. P is the only dimension-2 closure contained in them.

Proof. Three distinct states of $\mathbb{F}_2^3$ generate the four-state closure L. The five admitted states are the complement of L, which is also a closure, plus the fourth state of L. Uniqueness follows from Theorem 2 with $d = 2$ and $|S| = 5 < 6$. □

A fourth clause excluding exactly the pending state closes P; one excluding a state of P retracts it. So a formula of three-literal clauses produces a local closure with no parity equation written anywhere. Depending on the triple, that closure fixes one variable (24 of the 56 triples), equates or separates two (24), or is a parity of all three (8).

Recognizing the same closure when the relation is spread over many clauses is a different problem with its own cost: the subject of §7.

6. Counting when the reflections are given

Let F be the conjunction of parity equations $Ax = b$ over $\mathbb{F}_2$, with n variables, and disjunctive clauses $C_1, \dots, C_k$. In the thread:

- the equations are the inherited closure; if they are consistent and of rank r, they leave $d = n - r$ bits of free uncertainty;
- each clause moves the wall. If the closure already satisfies it ($Ax = b \vDash C_j$), it is inheritance and is removed without changing the solutions; otherwise it is novelty. There remain $t \le k$ novelty clauses.

Whether a clause is inherited is checked by Gaussian elimination: $Ax = b \wedge \neg C_j$ is inconsistent. Negating a clause amounts to setting its literals false, so every inclusion–exclusion term is another parity system. We write $N(\cdot)$ for the number of solutions:

$$N(F) = \sum_{J \subseteq \{1,\dots,k\}} (-1)^{|J|} N\Big(Ax = b \wedge \bigwedge_{j \in J} \neg C_j\Big).$$

Each term is 0 if the extended system is inconsistent and $2^{n - r_J}$ otherwise, with r_J its rank.

Theorem 4 (two exact routes). With the parity equations given, the number of solutions is computed in time

$$O(\,\mathrm{poly}(L) \cdot (1 + 2^{\min(t,d)})),$$

with L the input length, including building the basis and removing inherited clauses, in the usual model of operations on polynomial-size words. If the parity system is inconsistent, the number is zero.

Proof. Gaussian elimination builds the basis and tests every clause in polynomial time. With t clauses left, inclusion–exclusion visits at most 2^t subsets, each a linear system. Alternatively, enumerate the 2^d values of the free parameters and test every clause. Take the cheaper route. □

Read in the thread: the cost is paid in the smaller of two uncertainties, walking what the closure left free or unfolding the remaining novelty. It is polynomial when either is $O(\log L)$ and exponential when both grow linearly. When the input hides the reflections, the cost of recognizing them is added to the account.

7. The cost of recognizing what is inherited

The November 8 paper already measured the economy of observing, representing and executing. Here the prior question gets its own place: how much does it cost to know what was inherited before computing it again?

A recognizer E reads the formula and returns $E(F) = (R, \Delta, \pi)$: the recognized reference R, what is pending Δ, and a certificate π that lets one check the answer is rebuilt exactly. A checker V reviews it. The total cost is

$$C_{\text{total}} = C_E + C_V + C_{R,\Delta} + C_{\text{recon}}.$$

Transfer proposition (conditional). Suppose a family $\mathcal{F}$ admits a correct recognizer and checker, of costs $T_E(L)$ and $T_V(L)$; an exact decomposition of each formula into reference and pending part, disjoint or corrected by inclusion–exclusion; and a counter on (R, Δ) of cost $T_C(L, t, d)$. Then every $F \in \mathcal{F}$ is counted at cost $O(T_E + T_V + T_C + T_{\text{recon}})$. There is net economy against B when this sum is below the cost of B on the same family under the same model.

The proposition is a criterion for proving savings family by family. In an arbitrary three-literal formula, finding the best reference can require hard inference, and that cost enters the account in full.

The role of the mirror box is concrete: to propose changes of reference that preserve the answer, to separate inherited reflections from new differences, and to keep account of what reopens.

8. Controls with the reflections given

The controls in this section are in `experimentos/intercambio_y_convergencia/`. Times are those of its results file, measured with Python implementations.

Control	Result
Uniqueness of the reference and its construction (C5)	Residues mod 12 and 18 determine an integer within one cycle of 36, their least common multiple: in $[0, 72)$ each residue pair appears twice, and only 36 of the 216 possible pairs exist. Knowing the reference is unique and building it are two different jobs.
Recognizing before closing (C6)	Five of eight states: one four-state closure and one pending state; 336 continuations close and 1,344 retract.
Reflections given, favorable case (C7)	96 variables, 81 parity equations and 8 clauses: both routes count 10,656. Inclusion–exclusion visits 39 terms in 0.0015 s; variable elimination builds a 2^{24} table and takes 0.385 s.
Reflections given, unfavorable case (C7)	32 variables, 16 equations and 20 clauses: both count 5,220; inclusion–exclusion 0.0378 s, elimination 0.0021 s.
Much novelty, little closure (C7)	24 variables, 2 equations and 48 clauses: inclusion–exclusion passes the cap of two million terms; elimination counts 4,672, matching brute force.
Interpolation window (C8)	With $Q = 400$, a window of width $\sqrt{Q}$ leaves out 24.8% of the weight; of width $2\sqrt{Q}$, 2.1%; of width $3\sqrt{Q}$, 0.06%. A reference that is too narrow makes reconstruction expensive.

9. Reflections hidden in gates

Here the formula arrives only as clauses of at most three literals. The reflections, which are parity relations between primary variables, are built from two-input conjunction and disjunction gates and their auxiliary variables.

Variable names are permuted and clause order is shuffled. Some reflections arrive as a closure with one pending state. The recognizer identifies complete gates, rebuilds the parities and keeps the pending states.

Contract. The formula is a functional circuit of two-input gates, each assignment of the primary variables determines a single extension to the auxiliaries, and parities compose recognizably. Anything outside the contract is rejected.

Count-preservation proposition (under the contract). Replacing the verified gates by their parity relations, and each partial closure by its closure plus its pending state, preserves the number of solutions: primary assignments and their extensions are in bijection, and pending cases are added separately. With u partial closures and k novelty clauses, counting costs $O(\text{poly}(|F|) \cdot (1 + 2^{u+k}))$ on the family.

Measurement. The script `experimentos/linea_base_generica/comparacion_y_escala.py` measures every route on the same already-generated formulas, in the same run:

- recognize and count, median of 7;
- variable elimination on the formula as it arrives, recognizing nothing, median of 3, when its largest table is at most 2^{24};
- the laboratory DPLL, median of 3, up to 32 primary variables in the cycle and 24 with long connections.

All completed counts agree on every row. The long-connection family is extended up to 256 primary variables.

Family	Primaries	Clauses	Solutions	Recognize and count	Generic elimination	Largest elimination table	DPLL
cycle	24	248	5,312	1.4 ms	6.6 ms	2^6	227.7 ms
cycle	32	333	98,560	2.4 ms	9.4 ms	2^5	4,662 ms
cycle	96	941	420,906,795,008,000	12.9 ms	45.8 ms	2^6	—
long	24	409	26	2.6 ms	11.6 ms	2^9	19.6 ms
long	48	789	364	6.5 ms	27.8 ms	2^{13}	—
long	64	1,036	3,084	8.4 ms	54.9 ms	2^{18}	—
long	96	1,549	156,800	12.8 ms	247.0 ms	2^{21}	—
long	128	2,043	6,272,000	22.4 ms	—	2^{29}	—
long	200	3,183	25,690,112,000	41.3 ms	—	2^{40}	—
long	256	4,076	13,153,337,344,000	58.0 ms	—	2^{58}	—

All 23 measured formulas are in `RESULTADOS_COMPARACION.json`; the table shows ten.

Cycle of triples. Each reflection touches only its neighbors and the width is constant: elimination builds tables of at most 2^6. Both exact routes grow polynomially and differ by a factor of 3.5 to 5. The DPLL visits two nodes per solution (10,623 nodes for 5,312 solutions; 197,119 for 98,560): its time follows the number of solutions. In this family the advantage over DPLL comes from counting without enumerating, which both exact routes do.

Long connections. One third of the reflections join distant variables. The elimination width grows with size: 2^6 at 12 primaries, 2^{21} at 96 and 2^{58} at 256. A second ordering, minimum fill-in, gives 2^{19} and 2^{54}. Elimination counts up to 96 primaries, in 247 ms; from 128 on it needs tables of 2^{29} or more. Recognition counts all 256 primaries and 4,076 clauses in 58 ms. Here recognizing what is inherited pays: elimination stores as full tables relations that recognition resolves as parity equations.

Scope of the measurement. In all these formulas $u \leq 3$ and $k \leq 3$, so $2^{u+k} \leq 64$: the measurement follows the polynomial part of the bound. The next steps are to grow $u + k$ and to compare against dedicated counters that also recognize gates or use tree decompositions (§13). With `--dimacs`, the script writes each formula in DIMACS format with its expected count, ready for those counters.

Contract under test. `prueba_adversarial.py` takes valid formulas, applies one of seven random mutations (flipping a sign, removing or adding a clause, rewiring a literal, fixing a variable, swapping or identifying variables) and adds eight hand-built cases outside the family. Of 5,889 formulas, the recognizer rejects 4,444 and accepts 1,445, and all 1,445 accepted ones give the same count as generic elimination. The original laboratory also checks 144 gate truth tables, 30 random formulas against enumeration, and two out-of-contract controls.

10. Relation to the October preprints

Controls C1–C4 check, in representations built in the laboratory, pieces of two October 2026 preprints, one by OpenAI and one by Anthropic. The swap operator satisfies SWAP $= I - 2P_-$, with P_- the projection onto the antisymmetric part. With $r = 2$, the matrix V_0 of the OpenAI preprint equals the bitwise crossing of the Anthropic one for $\lambda = 1, \dots, 5$, and the per-piece rank formulas match the measured ranks at every tested size.

This is a local identity between representations. Whether a decomposition saves anything also depends on how many terms it produces and how much it costs to find and recombine them: the same account as in §2.

The program's question is a different one that complements them: when do reference and inheritance let us stop recomputing differences already resolved, while keeping what is pending.

11. Status of the results

Result	Status
Tetrahedron: six comparisons from three bits	Theorem 1
At most one closure of dimension d below the threshold $3 \cdot 2^{d-1}$	Theorem 2; existence checked for five states in $\mathbb{F}_2^3$
Three clauses: one four-state closure and one pending state	Theorem 3
Counting with given reflections in $O(\mathrm{poly}(L)(1 + 2^{\min(t,d)}))$	Theorem 4
Counting with reflections hidden in gates	Proposition under contract; 23 measured formulas; 5,889 formulas in the contract test
Savings against specific routes	Measured: favorable and unfavorable cases with given reflections (§8); constant factor at constant width and growing advantage with long connections (§9)
Dedicated counters, growth in $u + k$, lower bound for C_{rec}	Open
Net economy on three-literal formulas without recognizable templates	Open: central question (§12)

12. Questions for review

1. Refute a local proof with a state not considered.
2. Require semantic certificates for recognition when the reflections are spread out.
3. Find lower bounds for C_{rec}.
4. Count the exported formulas with dedicated counters (GANAK, SharpSAT-TD) and with definability preprocessing (B+E).
5. Build families where recognizing the minimal reference costs more than counting directly.

Central question. Is there a nontrivial family of three-literal formulas, without recognizable local templates, in which a checkable change of reference recovers an exact representation cheaper than the compared routes? A counterexample to any proposed mechanism also marks its scope.

13. Bridges to known work

The thread of this manuscript is written in the program's vocabulary. This table links it to the technical vocabulary.

In the thread	In the literature
comparison without direction in the tetrahedron	cut (cocycle) space of K_4, of dimension 3
closure	affine subspace of $\mathbb{F}_2^n$
closure with one pending state	affine root plus residue; parity block with three of its four clauses

In the thread	In the literature
given reflections	XOR equation system; Gaussian elimination
inheritance of a clause	implication $Ax = b \vDash C_j$
remaining novelty	OR clauses not implied
unfolding the novelty	inclusion–exclusion over clauses [Iwama 1989]
recognizing what is inherited	gate detection and definability [Lagniez and Marquis 2014; Lagniez, Lonca and Marquis 2016]
recognizer, checker and certificate	extractor, verifier and certificate
generic elimination	variable elimination; counting exponential in treewidth [Samer and Szeider 2010]
reflections inside a solver	native XOR with Gaussian elimination [Soos, Nohl and Castelluccia 2009]
dedicated counters	GANAK [Sharma et al. 2019]; SharpSAT-TD [Korhonen and Järvisalo 2021]

Count preservation under replacement of recognized gates (§9) has a direct precedent in B+E, which eliminates outputs of defined gates while preserving the number of models. What belongs to this synthesis is the sequence of the thread, built on known techniques: uncertainty that shrinks, recognition before closure, keeping what is pending, and a full account of the cost of recognizing.

References

From the program

- Endless Monkey Project, *¿Dónde se ahorra el cálculo? / Where does computation become cheaper?*, November 8, 2026.
- Endless Monkey Project, *El cruce como reflejo central*, November 3, 2026.
- Relational Geometry Program, *Focal reconstruction of the tetrahedron's links*, October 18, 2026.

External

- K. Iwama, "CNF-satisfiability test by counting and polynomial average time", *SIAM Journal on Computing* 18(2), 385–391, 1989.
- J.-M. Lagniez and P. Marquis, "Preprocessing for Propositional Model Counting", AAAI 2014, 2688–2694.
- J.-M. Lagniez, E. Lonca and P. Marquis, "Improving Model Counting by Leveraging Definability", IJCAI 2016, 751–757; "Definability for model counting", *Artificial Intelligence* 281, 2020.
- M. Samer and S. Szeider, "Algorithms for propositional model counting", *Journal of Discrete Algorithms* 8, 2010.
- M. Soos, K. Nohl and C. Castelluccia, "Extending SAT Solvers to Cryptographic Problems", SAT 2009, LNCS 5584, 244–257.
- S. Sharma, S. Roy, M. Soos and K. S. Meel, "GANAK: A Scalable Probabilistic Exact Model Counter", IJCAI 2019, 1169–1176.
- T. Korhonen and M. Järvisalo, "Integrating Tree Decompositions into Decision Heuristics of Propositional Model Counters", CP 2021, LIPIcs 210, 8:1–8:11.
- OpenAI, "The Exponential Time Hypothesis is False", preprint, October 25, 2026.
- Anthropic, "A $2^{o(n)}$ Time Algorithm for 3-SAT", Hexagon 2610.00577, October 27, 2026.

14. Reproduction

```
python3 -I experimentos/verificar_reconstruccion_y_convergencia.py
```

It checks Theorems 1 to 4 at small sizes, with no dependencies. Further instructions and the reproduction record are in `EVIDENCE.en.md`.